

UNIVERSIDAD NACIONAL DE COLOMBIA
Programación Orientada a Objetos 2026-2

ACTIVIDAD 2
Clases y objetos en Python con diagramas UML

Daniela Cataño Hernández
1018229103
dcatanoh@unal.edu.co

Walter Hugo Arboleda Mazo

8 de octubre de 2026

Ejercicio 2.1

Se requiere un programa que modele el concepto de una persona. Una persona posee nombre, apellidos, número de documento de identidad y año de nacimiento. La clase debe tener un constructor que inicialice los valores de sus atributos.

Código fuente (persona.py)

```
"""

class Persona:
    """Representa a una persona con sus datos básicos de identificación."""

    def __init__(self, nombre: str, apellidos: str,
                  numero_documento_identidad: str, anio_nacimiento: int) -> None:
        """
        Constructor de la clase Persona.

        :param nombre: nombre de la persona
        :param apellidos: apellidos de la persona
        :param numero_documento_identidad: número del documento de identidad
        :param anio_nacimiento: año de nacimiento de la persona
        """
        self.nombre = nombre
        self.apellidos = apellidos
        self.numero_documento_identidad = numero_documento_identidad
        self.anio_nacimiento = anio_nacimiento

    def imprimir(self) -> None:
        """Imprime en pantalla los valores de los atributos de la persona."""
        print(f"Nombre = {self.nombre}")
        print(f"Apellidos = {self.apellidos}")
        print(f"Número de documento de identidad = {self.numero_documento_identidad}")
        print(f"Año de nacimiento = {self.anio_nacimiento}")
        print()

def main() -> None:
    """Crea dos personas e imprime sus datos en pantalla."""
    p1 = Persona("Pedro", "Pérez", "1053121010", 1998)
    p2 = Persona("Luis", "León", "1053223344", 2001)
    p1.imprimir()
    p2.imprimir()

if __name__ == "__main__":
```

main()

Diagrama de clases

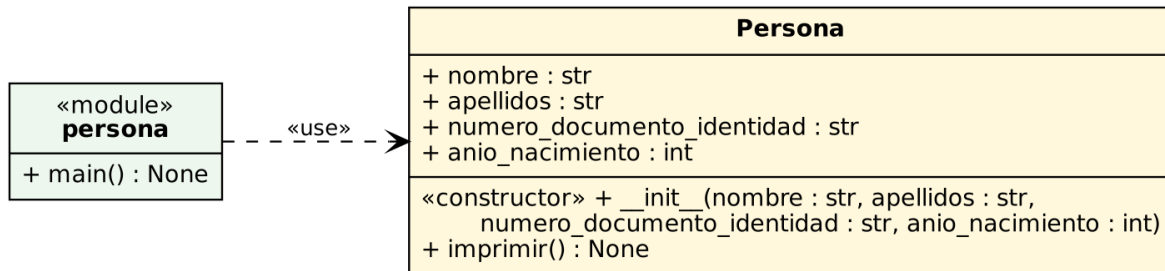


Figura 1. Diagrama de clases del ejercicio 2.1.

Explicación del diagrama de clase

Se definió una única clase, *Persona*. En el primer compartimiento está el nombre de la clase; en el segundo, sus cuatro atributos públicos (+) con su tipo: *nombre*, *apellidos* y *numero_documento_identidad* de tipo *str*, y *anio_nacimiento* de tipo *int*.

En el tercer compartimiento están los métodos: el constructor `__init__`, marcado con el estereotipo «constructor», y el método *imprimir*, que no recibe parámetros ni devuelve valor (*None*).

En Python la función *main* se define a nivel de módulo, por eso se representa con un elemento «module» (*persona*) que usa la clase *Persona* mediante una dependencia «use».

Ejecución del programa

Salida obtenida al ejecutar: `python persona.py`

```
... Nombre = Pedro
    Apellidos = Pérez
    Número de documento de identidad = 1053121010
    Año de nacimiento = 1998

    Nombre = Luis
    Apellidos = León
    Número de documento de identidad = 1053223344
    Año de nacimiento = 2001
```

URL del ejercicio:

<https://colab.research.google.com/drive/1Zy9o8dfLpKEv7C5t3S4Fnf4pO4dQFO8H?usp=sharing>

Ejercicio 2.2.

Se requiere un programa que modele un planeta del sistema solar con: nombre (str, inicial None), cantidad de satélites (int, 0), masa en kg (float, 0), volumen en km³ (float, 0), diámetro en km (int, 0), distancia media al Sol en millones de km (int, 0), tipo de planeta (enumerado: GASEOSO, TERRESTRE, ENANO) y si es observable a simple vista (bool, False).

Código fuente (planeta.py)

```
"""
```

Ejercicio 2.2 - Definición de atributos de una clase con tipos primitivos de datos.

```
"""
```

```
from enum import Enum
```

```
class TipoPlaneta(Enum):
```

```
    """Tipos de planeta de acuerdo con su tamaño."""
```

```
    GASEOSO = "Gaseoso"
```

```
    TERRESTRE = "Terrestre"
```

```
    ENANO = "Enano"
```

```
class Planeta:
```

```
    """Representa un planeta del sistema solar."""
```

```
    # Una unidad astronómica (UA) en kilómetros
```

```
    UA_KM: float = 149_597_870
```

```
    # Límite exterior del cinturón de asteroides, en UA
```

```
    LIMITE_CINTURON_UA: float = 3.4
```

```
    def __init__(self, nombre: str | None = None, cantidad_satelites: int = 0,
                  masa: float = 0.0, volumen: float = 0.0, diametro: int = 0,
                  distancia_sol: int = 0, tipo: TipoPlaneta | None = None,
                  es_observable: bool = False) -> None:
```

```
        """
```

Constructor de la clase Planeta. Los valores por defecto corresponden a los valores iniciales pedidos en el enunciado.

```
        :param nombre: nombre del planeta
```

```
        :param cantidad_satelites: cantidad de satélites
```

```

:param masa: masa en kilogramos
:param volumen: volumen en kilómetros cúbicos
:param diametro: diámetro en kilómetros
:param distancia_sol: distancia media al Sol en millones de kilómetros
:param tipo: tipo de planeta (GASEOSO, TERRESTRE o ENANO)
:param es_observable: si el planeta se observa a simple vista
"""

self.nombre = nombre
self.cantidad_satelites = cantidad_satelites
self.masa = masa
self.volumen = volumen
self.diametro = diametro
self.distancia_sol = distancia_sol
self.tipo = tipo
self.es_observable = es_observable

def imprimir(self) -> None:
    """Imprime en pantalla los valores de los atributos del planeta."""
    print(f"Nombre del planeta = {self.nombre}")
    print(f"Cantidad de satélites = {self.cantidad_satelites}")
    print(f"Masa del planeta = {self.masa:.4e} kg")
    print(f"Volumen del planeta = {self.volumen:.4e} km³")
    print(f"Diámetro del planeta = {self.diametro:,.} km")
    print(f"Distancia media al Sol = {self.distancia_sol} millones de km")
    print(f"Tipo de planeta = {self.tipo.value if self.tipo else None}")
    print(f"Es observable a simple vista = {'Sí' if self.es_observable else 'No'}")

def calcular_densidad(self) -> float:
    """
    Calcula la densidad del planeta como el cociente entre masa y volumen.

    :return: densidad en kg/km³ (0 si el volumen es cero)
    """
    if self.volumen == 0:
        return 0.0
    return self.masa / self.volumen

def es_planeta_exterior(self) -> bool:
    """
    Determina si el planeta está más allá del cinturón de asteroides
    (ubicado entre 2.1 y 3.4 UA).

    :return: True si el planeta es exterior
    """
    limite_millones_km = self.LIMITE_CINTURON_UA * self.UA_KM / 1_000_000
    return self.distancia_sol > limite_millones_km

```

```
def main() -> None:
    """Crea dos planetas e imprime sus datos, densidad y si son exteriores."""
    planetas = [
        Planeta("Tierra", 1, 5.9736e24, 1.08321e12, 12_742, 150,
            TipoPlaneta.TERRESTRE, True),
        Planeta("Júpiter", 79, 1.899e27, 1.4313e15, 139_820, 778,
            TipoPlaneta.GASEOSO, True),
    ]
    for planeta in planetas:
        planeta.imprimir()
        print(f"Densidad del planeta = {planeta.calcular_densidad():.4e} kg/km³")
        print(f"Es planeta exterior = {'Sí' if planeta.es_planeta_exterior() else 'No'}")
        print()

if __name__ == "__main__":
    main()
```

Diagrama de clases

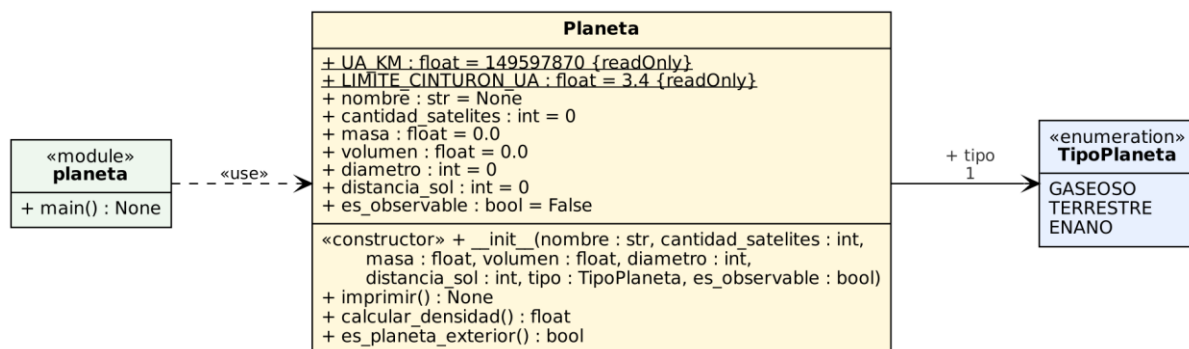


Figura 2. Diagrama de clases del ejercicio 2.2.

Explicación del diagrama de clases

La clase `Planeta` contiene sus atributos con el tipo y el valor inicial pedido en el enunciado; en Python los valores iniciales se expresan como valores por defecto de los parámetros del constructor.

Las constantes `UA_KM` y `LIMITE_CINTURON_UA` son atributos de clase (compartidos por todos los planetas), por eso aparecen subrayados y marcados como `{readOnly}`.

El tipo de planeta se modela como una asociación entre `Planeta` y la enumeración `TipoPlaneta` (estereotipo «enumeration»), cuyo nombre de rol es el atributo `tipo`. Los

métodos `calcular_densidad` y `es_planeta_exterior` devuelven `float` y `bool` respectivamente.

Mejora respecto al libro: la distancia se maneja de forma consistente en millones de km, tal como dice el enunciado, y el límite de 3.4 UA se convierte a esa misma unidad antes de comparar. Además, `calcular_densidad` evita la división por cero.

Ejecución del programa

```
... Nombre del planeta = Tierra
    Cantidad de satélites = 1
    Masa del planeta = 5.9736e+24 kg
    Volumen del planeta = 1.0832e+12 km³
    Diámetro del planeta = 12,742 km
    Distancia media al Sol = 150 millones de km
    Tipo de planeta = Terrestre
    Es observable a simple vista = Sí
    Densidad del planeta = 5.5147e+12 kg/km³
    Es planeta exterior = No

    Nombre del planeta = Júpiter
    Cantidad de satélites = 79
    Masa del planeta = 1.8990e+27 kg
    Volumen del planeta = 1.4313e+15 km³
    Diámetro del planeta = 139,820 km
    Distancia media al Sol = 778 millones de km
    Tipo de planeta = Gaseoso
    Es observable a simple vista = Sí
    Densidad del planeta = 1.3268e+12 kg/km³
    Es planeta exterior = Sí
```

URL del ejercicio:

https://colab.research.google.com/drive/1jg3_3Um2lpbR_UX5hdhp1Gi54qF45qU8?usp=sharing

Ejercicio 2.3

Se requiere un programa que modele un automóvil con: marca, modelo (año), motor (litros), tipo de combustible (gasolina, bioetanol, diésel, biodiésel, gas natural), tipo de automóvil (carro de ciudad, subcompacto, compacto, familiar, ejecutivo, SUV), número de puertas, cantidad de asientos, velocidad máxima, color (blanco, negro, rojo, naranja, amarillo, verde, azul, violeta) y velocidad actual.

Código fuente (automovil.py)

```
"""
```

```
from enum import Enum
```

```
class TipoCombustible(Enum):
```

```
    GASOLINA = "Gasolina"
```

```
    BIOETANOL = "Bioetanol"
```

```
    DIESEL = "Diésel"
```

```
    BODIESEL = "Biodiésel"
```

```
    GAS_NATURAL = "Gas natural"
```

```
class TipoAutomovil(Enum):
```

```
    CIUDAD = "Carro de ciudad"
```

```
    SUBCOMPACTO = "Subcompacto"
```

```
    COMPACTO = "Compacto"
```

```
    FAMILIAR = "Familiar"
```

```
    EJECUTIVO = "Ejecutivo"
```

```
    SUV = "SUV"
```

```
class Color(Enum):
```

```
    BLANCO = "Blanco"
```

```
    NEGRO = "Negro"
```

```
    ROJO = "Rojo"
```

```
    NARANJA = "Naranja"
```

```
    AMARILLO = "Amarillo"
```

```
    VERDE = "Verde"
```

```
    AZUL = "Azul"
```

```
    VIOLETA = "Violeta"
```

```
class Automovil:
```

```
    """Representa un automóvil cuyo estado cambia al acelerar o frenar."""
```

```
    def __init__(self, marca: str, modelo: int, motor: float,  
                  tipo_combustible: TipoCombustible, tipo_automovil: TipoAutomovil,
```

```

        numero_puertas: int, cantidad_asientos: int,
        velocidad_maxima: int, color: Color) -> None:
"""

```

Constructor de la clase Automovil. La velocidad actual inicia en 0.

```

:param marca: nombre del fabricante
:param modelo: año de fabricación
:param motor: cilindraje del motor en litros
:param tipo_combustible: tipo de combustible
:param tipo_automovil: tipo de automóvil
:param numero_puertas: cantidad de puertas
:param cantidad_asientos: número de asientos
:param velocidad_maxima: velocidad máxima en km/h
:param color: color del automóvil
"""

```

```

self._marca = marca
self._modelo = modelo
self._motor = motor
self._tipo_combustible = tipo_combustible
self._tipo_automovil = tipo_automovil
self._numero_puertas = numero_puertas
self._cantidad_asientos = cantidad_asientos
self._velocidad_maxima = velocidad_maxima
self._color = color
self._velocidad_actual = 0

```

----- Métodos get y set (propiedades) -----

```

@property
def marca(self) -> str:
    return self._marca

```

```

@marca.setter
def marca(self, valor: str) -> None:
    self._marca = valor

```

```

@property
def modelo(self) -> int:
    return self._modelo

```

```

@modelo.setter
def modelo(self, valor: int) -> None:
    self._modelo = valor

```

```

@property
def motor(self) -> float:
    return self._motor

```

```

@motor.setter
def motor(self, valor: float) -> None:
    self._motor = valor

@property
def tipo_combustible(self) -> TipoCombustible:
    return self._tipo_combustible

@tipo_combustible.setter
def tipo_combustible(self, valor: TipoCombustible) -> None:
    self._tipo_combustible = valor

@property
def tipo_automovil(self) -> TipoAutomovil:
    return self._tipo_automovil

@tipo_automovil.setter
def tipo_automovil(self, valor: TipoAutomovil) -> None:
    self._tipo_automovil = valor

@property
def numero_puertas(self) -> int:
    return self._numero_puertas

@numero_puertas.setter
def numero_puertas(self, valor: int) -> None:
    self._numero_puertas = valor

@property
def cantidad_asientos(self) -> int:
    return self._cantidad_asientos

@cantidad_asientos.setter
def cantidad_asientos(self, valor: int) -> None:
    self._cantidad_asientos = valor

@property
def velocidad_maxima(self) -> int:
    return self._velocidad_maxima

@velocidad_maxima.setter
def velocidad_maxima(self, valor: int) -> None:
    self._velocidad_maxima = valor

@property

```

```

def color(self) -> Color:
    return self._color

@color.setter
def color(self, valor: Color) -> None:
    self._color = valor

@property
def velocidad_actual(self) -> int:
    return self._velocidad_actual

@velocidad_actual.setter
def velocidad_actual(self, valor: int) -> None:
    """Asigna la velocidad validando que esté entre 0 y la máxima."""
    if 0 <= valor <= self._velocidad_maxima:
        self._velocidad_actual = valor
    else:
        print(f"Velocidad inválida: debe estar entre 0 y "
              f"{self._velocidad_maxima} km/h.")

# ----- Comportamiento -----
def acelerar(self, incremento: int) -> None:
    """Aumenta la velocidad sin superar la velocidad máxima."""
    if self._velocidad_actual + incremento <= self._velocidad_maxima:
        self._velocidad_actual += incremento
    else:
        print("No se puede incrementar a una velocidad superior "
              "a la máxima del automóvil.")

def desacelerar(self, decremento: int) -> None:
    """Disminuye la velocidad sin llegar a un valor negativo."""
    if self._velocidad_actual - decremento >= 0:
        self._velocidad_actual -= decremento
    else:
        print("No se puede decrementar a una velocidad negativa.")

def frenar(self) -> None:
    """Coloca la velocidad actual en cero."""
    self._velocidad_actual = 0

def calcular_tiempo_llegada(self, distancia: float) -> float | None:
    """
    Calcula el tiempo estimado de llegada (horas) para una distancia en km.

    :return: horas estimadas, o None si el automóvil está detenido
    """

```

```

if self._velocidad_actual == 0:
    print("El automóvil está detenido: no se puede estimar el tiempo de llegada.")
    return None
return distancia / self._velocidad_actual

```

```

def imprimir(self) -> None:
    """Imprime en pantalla los valores de los atributos del automóvil."""
    print(f"Marca = {self._marca}")
    print(f"Modelo = {self._modelo}")
    print(f"Motor = {self._motor} L")
    print(f"Tipo de combustible = {self._tipo_combustible.value}")
    print(f"Tipo de automóvil = {self._tipo_automobil.value}")
    print(f"Número de puertas = {self._numero_puertas}")
    print(f"Cantidad de asientos = {self._cantidad_asientos}")
    print(f"Velocidad máxima = {self._velocidad_maxima} km/h")
    print(f"Color = {self._color.value}")
    print(f"Velocidad actual = {self._velocidad_actual} km/h")

```

```

def main() -> None:
    """Crea un automóvil y realiza varios cambios de velocidad."""
    auto1 = Automovil("Ford", 2018, 3.0, TipoCombustible.DIESEL,
                      TipoAutomovil.EJECUTIVO, 5, 6, 250, Color.NEGRO)
    auto1.imprimir()
    print()

    auto1.velocidad_actual = 100
    print(f"Velocidad actual = {auto1.velocidad_actual} km/h")
    tiempo = auto1.calcular_tiempo_llegada(300)
    print(f"Tiempo estimado para recorrer 300 km = {tiempo:.2f} h")

    auto1.acelerar(20)
    print(f"Velocidad actual (tras acelerar 20) = {auto1.velocidad_actual} km/h")

    auto1.desacelerar(50)
    print(f"Velocidad actual (tras desacelerar 50) = {auto1.velocidad_actual} km/h")

    auto1.frenar()
    print(f"Velocidad actual (tras frenar) = {auto1.velocidad_actual} km/h")

    # Casos no permitidos: muestran el mensaje correspondiente
    auto1.desacelerar(20)
    auto1.acelerar(300)

```

```

if __name__ == "__main__":

```

main()

Diagrama de clases

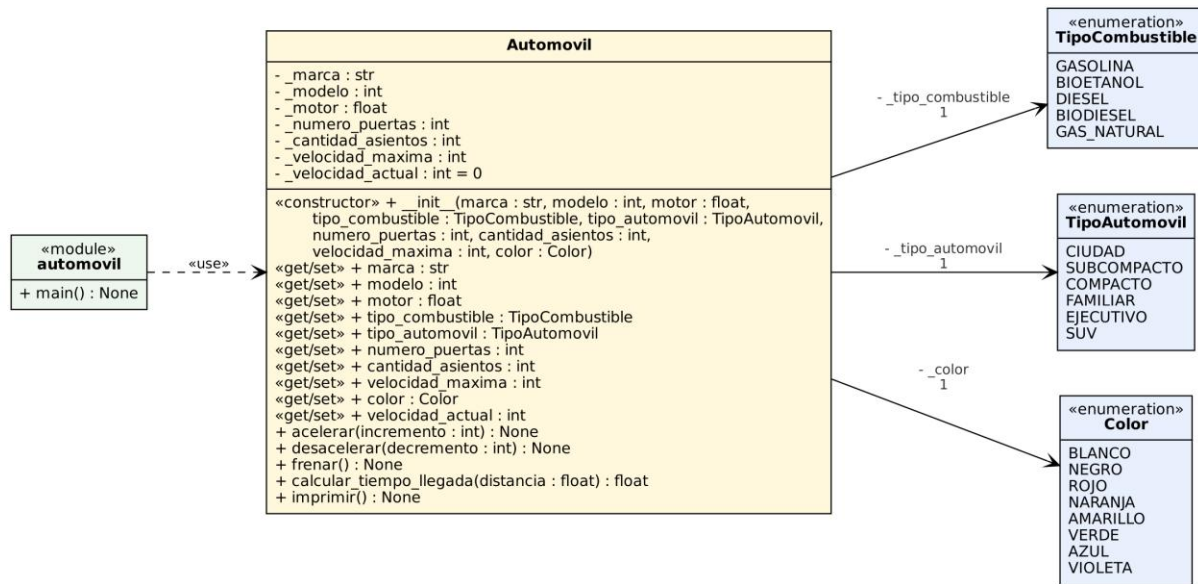


Figura 3. Diagrama de clases del ejercicio 2.3.

Explicación del diagrama de clases

Los atributos de Automovil son privados (prefijo `_` en Python, visibilidad `-` en UML), de modo que el estado del objeto solo cambia a través de sus métodos. Los métodos `get` y `set` se implementan con propiedades (`@property` y `.setter`), la forma idiomática de Python; en el diagrama se identifican con el estereotipo `«get/set»`.

Los tres atributos enumerados se modelan como asociaciones con las enumeraciones `TipoCombustible`, `TipoAutomovil` y `Color`, cada una con sus constantes en el compartimiento de atributos.

Los métodos `acelerar`, `desacelerar` y `frenar` modifican el estado (la velocidad actual) y validan los límites: no se puede superar la velocidad máxima ni quedar en velocidad negativa; en esos casos se muestra un mensaje. El `set` de `velocidad_actual` también valida el rango.

Mejora respecto al libro: `calcular_tiempo_llegada` evita la división por cero cuando el automóvil está detenido y usa división real (en Java, `int/int` perdía los decimales). `Acelerar` exactamente hasta la velocidad máxima sí está permitido.

Ejecución del programa

```
Marca = Ford
Modelo = 2018
Motor = 3.0 L
Tipo de combustible = Diésel
Tipo de automóvil = Ejecutivo
Número de puertas = 5
Cantidad de asientos = 6
Velocidad máxima = 250 km/h
Color = Negro
Velocidad actual = 0 km/h

Velocidad actual = 100 km/h
Tiempo estimado para recorrer 300 km = 3.00 h
Velocidad actual (tras acelerar 20) = 120 km/h
Velocidad actual (tras desacelerar 50) = 70 km/h
Velocidad actual (tras frenar) = 0 km/h
No se puede decrementar a una velocidad negativa.
No se puede incrementar a una velocidad superior a la máxima del automóvil.
```

URL del ejercicio:

https://colab.research.google.com/drive/1ig3_3Um2lpbR_UX5hdhp1Gi54qF45qU8?usp=sharing

Ejercicio 2.4

Se requiere un programa que modele varias figuras geométricas: el círculo (radio), el rectángulo (base y altura), el cuadrado (lado) y el triángulo rectángulo (base y altura), todas en centímetros.

Se requieren métodos para calcular el área y el perímetro de cada figura. Para el triángulo rectángulo, además, un método que calcule la hipotenusa y otro que determine si es equilátero, isósceles o escaleno. Se desarrolla un programa de prueba con main que crea las cuatro figuras y prueba sus métodos.

Código fuente (figuras.py)

```
"""
```

Ejercicio 2.4 - Definición de métodos con y sin valores de retorno.

```
"""
```

```
import math
```

```
class Circulo:
```

```
    """Círculo definido por su radio."""
```

```
    def __init__(self, radio: float) -> None:
        self.radio = radio
```

```
    def calcular_area(self) -> float:
        """Área =  $\pi \cdot r^2$ """
        return math.pi * self.radio ** 2
```

```
    def calcular_perimetro(self) -> float:
        """Perímetro =  $2 \cdot \pi \cdot r$ """
        return 2 * math.pi * self.radio
```

```
class Rectangulo:
```

```
    """Rectángulo definido por su base y su altura."""
```

```
    def __init__(self, base: float, altura: float) -> None:
        self.base = base
        self.altura = altura
```

```
    def calcular_area(self) -> float:
        """Área = base · altura"""
        return self.base * self.altura
```

```
    def calcular_perimetro(self) -> float:
```

```

        """Perímetro = 2 · base + 2 · altura"""
        return 2 * self.base + 2 * self.altura

```

class Cuadrado:

```

    """Cuadrado definido por la longitud de su lado."""

```

```

    def __init__(self, lado: float) -> None:
        self.lado = lado

```

```

    def calcular_area(self) -> float:
        """Área = lado2"""
        return self.lado ** 2

```

```

    def calcular_perimetro(self) -> float:
        """Perímetro = 4 · lado"""
        return 4 * self.lado

```

class TrianguloRectangulo:

```

    """Triángulo rectángulo definido por su base y su altura (los catetos)."""

```

```

    def __init__(self, base: float, altura: float) -> None:
        self.base = base
        self.altura = altura

```

```

    def calcular_area(self) -> float:
        """Área = base · altura / 2"""
        return self.base * self.altura / 2

```

```

    def calcular_hipotenusa(self) -> float:
        """Hipotenusa por el teorema de Pitágoras:  $\sqrt{(base^2 + altura^2)}$ """
        return math.hypot(self.base, self.altura)

```

```

    def calcular_perimetro(self) -> float:
        """Perímetro = base + altura + hipotenusa"""
        return self.base + self.altura + self.calcular_hipotenusa()

```

```

    def determinar_tipo_triangulo(self) -> str:
        """
        Determina el tipo de triángulo según sus lados:
        equilátero (3 iguales), isósceles (2 iguales) o escaleno (todos distintos).

        :return: nombre del tipo de triángulo
        """
        a, b, c = self.base, self.altura, self.calcular_hipotenusa()

```

```

iguales = sum([math.isclose(a, b), math.isclose(a, c), math.isclose(b, c)])
if iguales == 3:
    return "Equilátero"
if iguales >= 1:
    return "Isósceles"
return "Escaleno"

```

```

def main() -> None:

```

```

    """Clase de prueba: crea las cuatro figuras y prueba sus métodos."""

```

```

    figuras = {
        "círculo (radio 2)": Circulo(2),
        "rectángulo (base 1, altura 2)": Rectangulo(1, 2),
        "cuadrado (lado 3)": Cuadrado(3),
        "triángulo rectángulo (base 3, altura 5)": TrianguloRectangulo(3, 5),
    }

```

```

    for nombre, figura in figuras.items():
        print(f"El área del {nombre} es = {figura.calcular_area():.2f} cm²")
        print(f"El perímetro del {nombre} es = {figura.calcular_perimetro():.2f} cm")
        print()

```

```

    triangulo = figuras["triángulo rectángulo (base 3, altura 5)"]
    print(f"La hipotenusa del triángulo es = {triangulo.calcular_hipotenusa():.2f} cm")
    print(f"El triángulo es {triangulo.determinar_tipo_triangulo()}")

```

```

    otro = TrianguloRectangulo(4, 4)
    tipo = otro.determinar_tipo_triangulo()
    print(f"Un triángulo rectángulo de base 4 y altura 4 es {tipo}")

```

```

if __name__ == "__main__":

```

```

    main()

```

Diagrama de clases

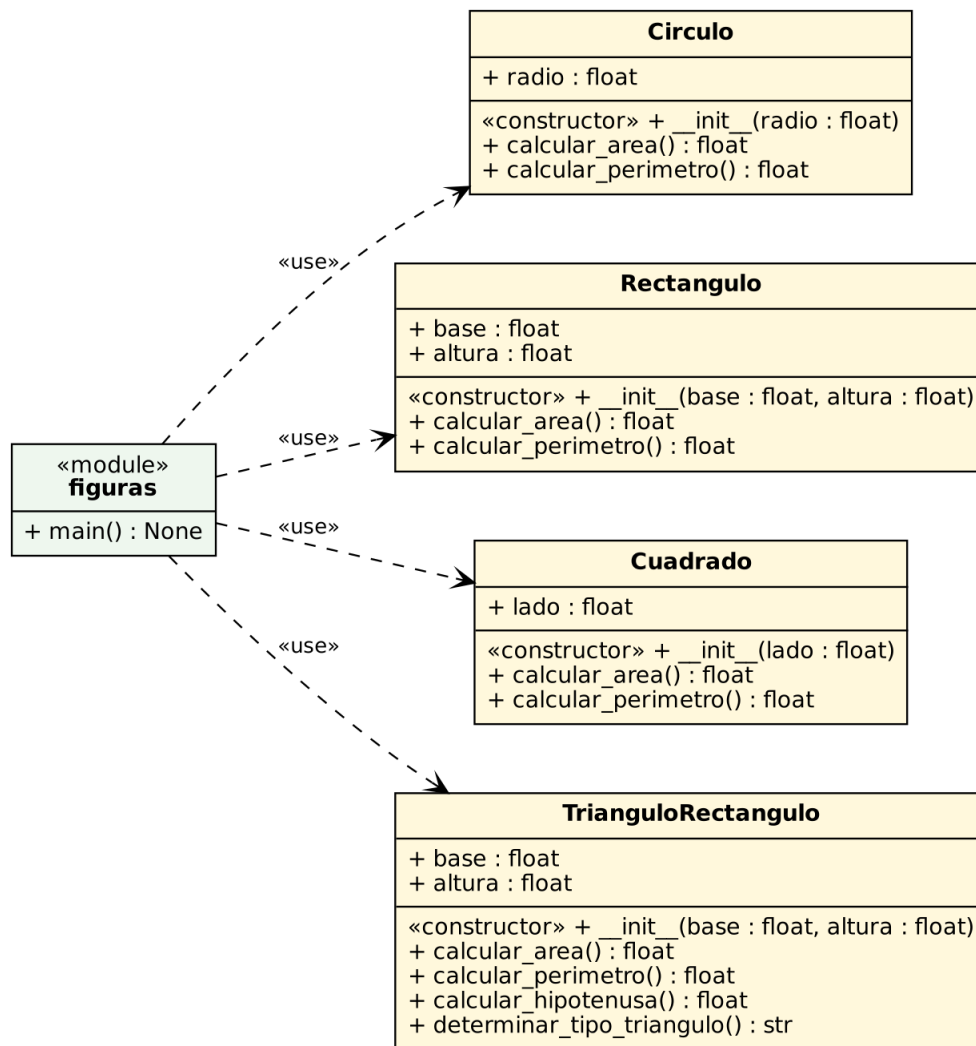


Figura 4. Diagrama de clases del ejercicio 2.4.

Explicación del diagrama de clases

Se definió una clase por figura: **Circulo**, **Rectangulo**, **Cuadrado** y **TrianguloRectangulo**, cada una con sus atributos (float) y un constructor. Todas tienen los métodos **calcular_area** y **calcular_perimetro**, que devuelven float.

TrianguloRectangulo tiene dos métodos adicionales: **calcular_hipotenusa** (teorema de Pitágoras) y **determinar_tipo_triangulo**, que devuelve el nombre del tipo de triángulo (str). La hipotenusa no es un atributo sino un valor calculado.

El módulo **figuras** cumple el papel de la clase de prueba **PruebaFiguras** del libro: contiene la función **main** y depende («use») de las cuatro clases.

Mejora respecto al libro: `determinar_tipo_triangulo` devuelve el resultado en lugar de imprimirlo y compara los lados con tolerancia (`math.isclose`), ya que la hipotenusa es un número real. El área del triángulo usa división real (en Java, `base*altura/2` con enteros truncaba 7.5 a 7).

Ejecución del programa

```
El área del círculo (radio 2) es = 12.57 cm2
El perímetro del círculo (radio 2) es = 12.57 cm

El área del rectángulo (base 1, altura 2) es = 2.00 cm2
El perímetro del rectángulo (base 1, altura 2) es = 6.00 cm

El área del cuadrado (lado 3) es = 9.00 cm2
El perímetro del cuadrado (lado 3) es = 12.00 cm

El área del triángulo rectángulo (base 3, altura 5) es = 7.50 cm2
El perímetro del triángulo rectángulo (base 3, altura 5) es = 13.83 cm

La hipotenusa del triángulo es = 5.83 cm
El triángulo es Escaleno
Un triángulo rectángulo de base 4 y altura 4 es Isósceles
```

URL del ejercicio:

<https://colab.research.google.com/drive/15bUNBKAVA4zR7vw0TeurxcrG3DXQmQ-I?usp=sharing>

Ejercicio 2.5

Se requiere un programa que modele una cuenta bancaria con: nombres del titular, apellidos del titular, número de la cuenta, tipo de cuenta (ahorros o corriente) y saldo. El constructor inicializa los atributos y el saldo inicial es cero.

Código fuente (cuenta_bancaria.py)

```
"""
Ejercicio 2.5 - Definición de métodos con parámetros.
"""

from enum import Enum

class TipoCuenta(Enum):
    AHORROS = "Ahorros"
    CORRIENTE = "Corriente"

class CuentaBancaria:
    """Representa una cuenta bancaria. El saldo inicia en cero."""

    def __init__(self, nombres_titular: str, apellidos_titular: str,
                 numero_cuenta: str, tipo_cuenta: TipoCuenta) -> None:
        """
        Constructor de la clase CuentaBancaria. El saldo no se recibe como
        parámetro porque toda cuenta nueva inicia con saldo cero.

        :param nombres_titular: nombres del titular
        :param apellidos_titular: apellidos del titular
        :param numero_cuenta: número de la cuenta
        :param tipo_cuenta: AHORROS o CORRIENTE
        """

        self.nombres_titular = nombres_titular
        self.apellidos_titular = apellidos_titular
        self.numero_cuenta = numero_cuenta
        self.tipo_cuenta = tipo_cuenta
        self._saldo = 0.0 # Encapsulado: solo cambia con consignar/retirar

    @property
    def saldo(self) -> float:
        """Saldo actual (solo lectura)."""
        return self._saldo

    def imprimir(self) -> None:
        """Imprime en pantalla los datos de la cuenta bancaria."""
```

```

print(f"Nombres del titular = {self.nombres_titular}")
print(f"Apellidos del titular = {self.apellidos_titular}")
print(f"Número de cuenta = {self.numero_cuenta}")
print(f"Tipo de cuenta = {self.tipo_cuenta.value}")
print(f"Saldo = ${self._saldo:,.2f}")

```

```

def consultar_saldo(self) -> None:
    """Imprime en pantalla el saldo actual."""
    print(f"El saldo actual es = ${self._saldo:,.2f}")

```

```

def consignar(self, valor: float) -> bool:
    """
    Consigna un valor en la cuenta y actualiza el saldo.

    :param valor: valor a consignar (debe ser mayor que cero)
    :return: True si la consignación se realizó
    """
    if valor <= 0:
        print("El valor a consignar debe ser mayor que cero.")
        return False
    self._saldo += valor
    print(f"Se ha consignado ${valor:,.2f}. El nuevo saldo es ${self._saldo:,.2f}")
    return True

```

```

def retirar(self, valor: float) -> bool:
    """
    Retira un valor de la cuenta y actualiza el saldo.

    :param valor: valor a retirar (mayor que cero y no mayor al saldo)
    :return: True si el retiro se realizó
    """
    if valor <= 0:
        print("El valor a retirar debe ser mayor que cero.")
        return False
    if valor > self._saldo:
        print(f"Fondos insuficientes: no se puede retirar ${valor:,.2f} "
              f"con un saldo de ${self._saldo:,.2f}")
        return False
    self._saldo -= valor
    print(f"Se ha retirado ${valor:,.2f}. El nuevo saldo es ${self._saldo:,.2f}")
    return True

```

```

def main() -> None:
    """Crea una cuenta y realiza consignaciones y retiros."""
    cuenta = CuentaBancaria("Pedro", "Pérez", "123456789", TipoCuenta.AHORROS)

```

```

cuenta.imprimir()
print()
cuenta.consignar(200_000)
cuenta.consignar(300_000)
cuenta.retirar(400_000)
cuenta.retirar(500_000) # No permitido: supera el saldo
cuenta.consignar(-1_000) # No permitido: valor negativo
cuenta.consultar_saldo()

```

```
if __name__ == "__main__":
```

```
    main()
```

Diagrama de clases

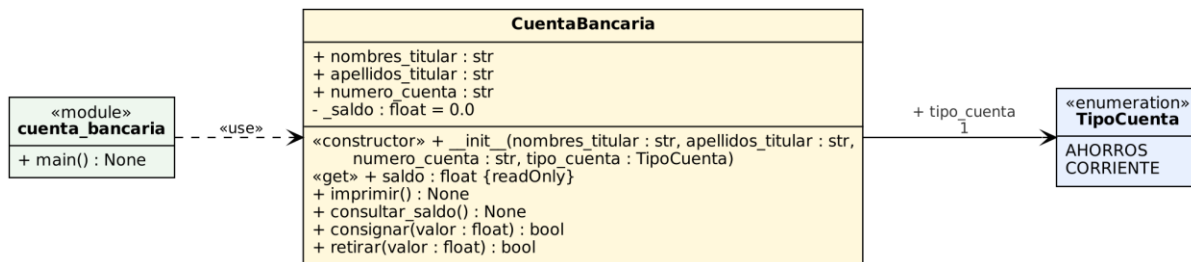


Figura 5. Diagrama de clases del ejercicio 2.5.

Explicación del diagrama de clases

La clase `CuentaBancaria` tiene como atributos públicos los nombres y apellidos del titular y el número de cuenta (str). El saldo es privado (`_saldo`) con valor inicial 0.0 y se expone mediante la propiedad de solo lectura `saldo`, de modo que solo puede cambiar a través de `consignar` y `retirar`.

El tipo de cuenta es un valor enumerado, representado por la asociación con la enumeración `TipoCuenta` (AHORROS, CORRIENTE).

Los métodos `consignar` y `retirar` reciben un parámetro (el valor) y devuelven un bool que indica si la operación se pudo realizar. `imprimir` y `consultar_saldo` no reciben parámetros ni devuelven valor.

Mejora respecto al libro: el número de cuenta se guarda como texto (puede tener ceros a la izquierda y no se usa para cálculos), `retirar` distingue entre valor no positivo y fondos insuficientes, y el saldo queda protegido contra asignaciones directas.

Ejecución del programa

```
Nombres del titular = Pedro
Apellidos del titular = Pérez
Número de cuenta = 123456789
Tipo de cuenta = Ahorros
Saldo = $0.00

Se ha consignado $200,000.00. El nuevo saldo es $200,000.00
Se ha consignado $300,000.00. El nuevo saldo es $500,000.00
Se ha retirado $400,000.00. El nuevo saldo es $100,000.00
Fondos insuficientes: no se puede retirar $500,000.00 con un saldo de $100,000.00
El valor a consignar debe ser mayor que cero.
El saldo actual es = $100,000.00
```

URL del ejercicio

https://colab.research.google.com/drive/1EHkRZIAZ2b4_4kPBZ2anC5Bns1vbhjTU?usp=sharing